# Lab 3 Component Modelling and Architecture

## Internal Microservice Catalog & Health Portal

Name: Rufus Thomas
SRN:- PES1UG24AM232

## Architecture justification

We chose Microservices Architecture for the Internal Microservice Catalog & Health Portal. The portal is organized as independently deployable services for catalog management, health monitoring, alert management, dependency-graph visualization, and API documentation, with shared data and external integrations. The scenario explicitly centers on mapping microservice dependencies, aggregating API documentation, and running periodic health-check pingers with downtime alerts.

Scenario and challenges: The system maintains a service catalog, displays dependency relationships, aggregates API documentation, probes service health every 30 seconds, computes rolling 24-hour availability, and dispatches alerts after three consecutive failed probes. Its main challenges are continuous monitoring load, accurate availability calculation, dependency-graph responsiveness for up to 200 services, secure access to infrastructure information, notification reliability, and handling unreachable services without incorrectly marking them healthy.

Style comparison: Layered architecture gives clear separation and simpler maintenance, but scaling a continuously running health-monitoring workload independently is harder. Client-server provides centralized control and simple deployment, but the server becomes a shared scaling and failure point. Microservices fit the domain because monitoring, catalog, documentation, graph, and alert workloads can be deployed, isolated, and scaled separately.

Reason 1 for the choice: The health-monitoring workload is periodic and potentially intensive. Keeping Health Monitor independent lets it scale without forcing unrelated catalog and documentation functions to scale with it.

Reason 2 for the choice: The portal has distinct responsibilities with clear service boundaries. Dependency graphs, API documentation, service registration, and alerting can evolve independently while communicating through explicit interfaces.

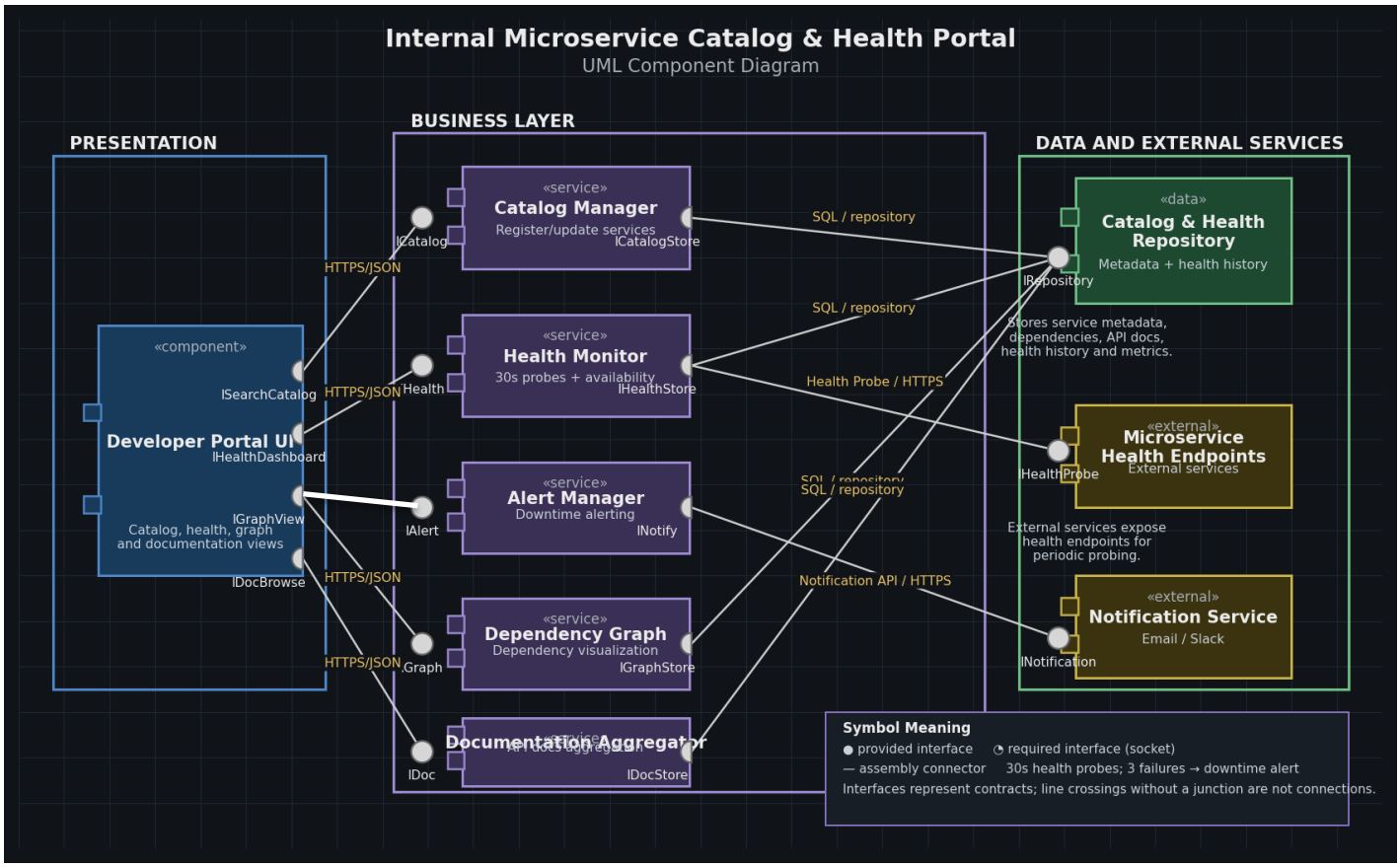
Security advantage: Access to the portal can be authenticated and authorized centrally, while each service exposes only the endpoints it needs. Health data and infrastructure metadata remain behind service APIs, and credentials for external integrations are held server-side rather than in the browser.

Performance benefit: Services can scale independently and monitoring can run concurrently. The dependency-graph viewer can be scaled for larger architectures, while the Health Monitor handles 30-second probes without blocking catalog or documentation requests.

Drawbacks: Microservices introduce network latency, distributed tracing and monitoring overhead, deployment complexity, and possible consistency challenges. The design should therefore keep service contracts narrow and avoid unnecessary distributed transactions.

# UML component diagram

## Internal Microservice Catalog & Health Portal




Created and exported as a UML component design. Five core business services, one presentation component, one repository, and two external providers; named interfaces show provided and required contracts.

# Components and interface contracts

The diagram is a proposed design for the Internal Microservice Catalog & Health Portal. Interfaces show service contracts rather than requiring a separate deployment for every boundary. The Catalog & Health Repository encapsulates persistence; Microservice Health Endpoints and Notification Service are outside the application boundary.

## Component Responsibility

| Component | Responsibility |
|---|---|
| **Developer Portal UI** | Catalog, health, dependency-graph and documentation views; supports DevOps Engineer and System Architect workflows. |
| **Catalog Manager** | Registers, updates and retrieves microservice metadata, ownership, endpoints and versions. |
| **Health Monitor** | Pings health endpoints every 30 seconds, records failures, and computes rolling 24-hour availability. |
| **Alert Manager** | Applies the three-consecutive-failure rule and dispatches downtime notifications. |
| **Dependency Graph Service** | Builds and renders interactive service-dependency relationships. |
| **Documentation Aggregator** | Collects and exposes API documentation for registered services. |
| **Catalog & Health Repository** | Persists service metadata, dependency mappings, API documentation references, health history and availability metrics. |
| **Microservice Health Endpoints** | External service endpoints queried by the Health Monitor. |
| **Notification Service** | External email/Slack-style notification provider used for downtime alerts. |

## Interface consumers and providers

| Interface | Consumer to provider | Request and returned result |
|---|---|---|
| **ISearchCatalog** | Portal UI → Catalog Manager | Search/register/update service; service metadata or validation result. |
| **IHealthDashboard** | Portal UI → Health Monitor | Read health status and availability; current status and 24-hour metric. |
| **IGraphView** | Portal UI → Dependency Graph | Request dependency graph; interactive service graph. |
| **IDocBrowse** | Portal UI → Documentation Aggregator | Browse API docs; aggregated documentation. |
| **ICatalogStore** | Catalog Manager → Repository | Persist/read service metadata; authorized catalog data. |
| **IHealthStore** | Health Monitor → Repository | Store probes and availability history; saved health record. |
| **IGraphStore** | Dependency Graph → Repository | Read dependency relationships; graph data. |
| **IDocStore** | Documentation Aggregator → Repository | Read documentation references/metadata; aggregated documentation data. |
| **IHealthProbe** | Health Monitor → Microservice Endpoints | Probe service endpoint; health response or timeout/failure. |
| **INotify** | Alert Manager → Notification Service | Dispatch downtime alert; delivery result. |

**Example interaction.** A DevOps Engineer searches the catalog through ISearchCatalog. The portal requests service data from Catalog Manager, which reads the repository through ICatalogStore. In parallel, Health Monitor performs its 30-second probes through IHealthProbe and records results through IHealthStore. After three consecutive failed probes, Alert Manager uses INotify to dispatch a downtime notification. The portal can then display the updated service status and rolling 24-hour availability.